



Conteúdo

1 DP

1.1	Knapsack	1
1.2	LIS	1
1.3	LCS	1
1.4	Subset-sum	1

2 Annotation

2.1	Annotation	1
-----	------------	---

3 Graphs

3.1	DSU	1
3.2	Dijkstra	1
3.3	Floyd-Warshall	1
3.4	Bellman-Ford	1
3.5	Kruskal	1

4 Range Queries

4.1	Merge sort	1
4.2	Count inversions (Divide and Conquer)	1
4.3	Segment Tree (range min)	1
4.4	Fenwick Tree	1
4.5	Count inversions (Fenwick + compression)	1
4.6	K-th query on frequency SegTree	1
4.7	Binary search on SegTree	1
4.8	Subarray Sum Queries	1

1 DP

1.1 Knapsack

```
vector<int> w, v;
// W = knaps capacity, n = n items
// w[i] = weight, v[i] = value
int knapsack(int n, int W) {
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int k = W - w[i]; k >= 0; k--) {
            dp[k + w[i]] = max(dp[k + w[i]], dp[k] + v[i]);
        }
    }

    return dp[W];
}
```

1.2 LIS

```
int lis(vector<int>& nums) {
    vector<int> v;

    for (auto x: nums) {
        auto it = lower_bound(v.begin(), v.end(), x);
        // lower -> a > b
        // upper -> a >= b (allow duplicates)

        if (it == v.end()) v.push_back(x);
        else *it = x;
    }
}
```

```
return v.size();
}
```

1.3 LCS

```
1
1 int lcs(string a, string b) {
1     vector<vector<int>> m(a.size()+1, vector<int>(b.size()+1));
1     for (int i = a.size(); i >= 0; i--) {
1         for (int j = b.size(); j >= 0; j--) {
1             if (i == a.size() || j == b.size()) {
1                 m[i][j] = 0; continue;
1             }

2             if (a[i] == b[j]) m[i][j] = 1 + m[i+1][j+1];
2             else m[i][j] = max(m[i][j+1], m[i+1][j]);
2         }
2     }

2     return m[0][0]; // result
2 }
```

1.4 Subset-sum

```
3
3 vector<int> w, v;
3 vector<vector<int>> memo;
4
4 // max sum(v) with sum(w) <= remW
4 int subsum(int id, int remW) {
5     if ((id == w.size()) || (remW == 0)) return 0;
5     int &ans = memo[id][remW];
5     if (ans != -1) return ans;
5     if (w[id] > remW) return ans = subsum(id+1, remW);
5     return ans = max(subsum(id + 1, remW),
5                       v[id]+subsum(id+1, remW-w[id]));
5 }

// which values can be created from a list a (int)
int subsumi(int n, int max_v, vector<int> a) {
    vector<int> m(max_v + 10, 0);

    m[0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = max_v; j >= a[i]; j--) {
            m[j] |= m[j - a[i]];
        }
    }

    return m[max_v];
}
```

2 Annotation

2.1 Annotation

```
g++ "$1.cpp" -o "$1" -Wall -Wextra -O2 -Wshadow -fsanitize=address,undefined
```

3 Graphs

3.1 DSU

```
struct DSU {
    vector<int> parent, size;
    DSU(int n) : parent(n+1), size(n+1) {
        for(int i=0; i<=n; i++) {parent[i] = i; size[i] = 1;}
    }

    int find(int u) {
        if (parent[u] == u) return u;

        return parent[u] = find(parent[u]);
    }

    void join(int u, int v) {
        u = find(u); v = find(v);
        if (u == v) return;

        if (size[u] < size[v]) swap(u, v);
        parent[v] = u;
        size[u] += size[v];
    }
};
```

3.2 Dijkstra

```
vector<pii> g[MAXN];

void dijkstra(int s, int n, vector<ll>& d) {
    d.assign(n, INF);

    priority_queue<pii, vector<pii>, greater<pii>> q;
    d[s] = 0; q.push({0, s});

    while (!q.empty()) {
        auto [d_u, u] = q.top(); q.pop();
        if (d_u > d[u]) continue;

        for (auto [v, w] : g[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                q.push({d[v], v});
            }
        }
    }
}
```

3.3 Floyd-Warshall

```
bool floyd_warshall(int n, vector<vector<pii>>& g) {
    vector<vector<ll>> d(n);
    d.assign(n, vector<ll>(n, INF));
    rep(i, n) d[i][i] = 0;
    rep(i, n) for (auto [j,w]: g[i]) d[i][j] = w;

    rep(k, n)
    rep(i, n)
    rep(j, n)
        if (d[i][k] < INF && d[k][j] < INF)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    rep(i, n)
```

```
        if (d[i][i] < 0) return true; // cycle

    return false;
}
```

3.4 Bellman-Ford

```
vector<tiit> edges;

void bellman_ford(int s, int n, vector<ll>& dist) {
    dist.assign(n, INF);
    dist[s] = 0;
    vector<int> p(n, -1);

    int x = -1; // cycle detect
    for (int i = 0; i < n; i++) {
        x = -1;
        for (int j = 0; j < edges.size(); j++) {
            auto [u, v, w] = edges[j];
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                x = v; p[v] = u;
            }
        }
    }

    if (x == -1) {} // no cycle
    else {
        int y = x;
        for (int i = 0; i < n; i++)
            y = p[y];

        vector<int> cycle;
        for (int cur = y;; cur = p[cur]) {
            cycle.push_back(cur);
            if (cur == y && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end()); // negative cycle
    }
}
```

3.5 Kruskal

```
struct edge {int u, v, w;};
bool comp(edge a, edge b) {return a.w < b.w;};

vector<edge> edges;

int kruskal(int n) {
    DSU ds(n);
    sort(edges.begin(), edges.end(), comp);

    int cost;
    for (auto [u, v, w] : edges) {
        if (ds.find(u) != ds.find(v)) {
            cost += w;
            ds.join(u, v);
        }
    }

    return cost;
}
```

4 Range Queries

4.1 Merge sort

```
void merge_sort(vector<int>& a, int l, int r) {
    if (l >= r) return;

    int mid = (l + r) / 2;
    merge_sort(a, l, mid);
    merge_sort(a, mid + 1, r);

    vector<int> temp;
    temp.reserve(r - l + 1);

    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else temp.push_back(a[j++]);
    }

    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }
}
```

4.2 Count inversions (Divide and Conquer)

```
ll merge_count(vector<int>& a, int l, int r) {
    if (l >= r) return 0;

    int mid = (l + r) / 2;
    ll inv = merge_count(a, l, mid) + merge_count(a, mid + 1, r);

    vector<int> temp;
    temp.reserve(r - l + 1);

    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else {
            inv += (mid - i + 1);
            temp.push_back(a[j++]);
        }
    }

    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }

    return inv;
}

ll count_inversions(vector<int> a) {
    if (a.empty()) return 0;
    return merge_count(a, 0, (int)a.size() - 1);
}
```

4.3 Segment Tree (range min)

```
struct SegTreeMin {
    int n;
    vector<ll> st;

    SegTreeMin() {}
    SegTreeMin(const vector<ll>& a) { init(a); }

    void init(const vector<ll>& a) {
        n = (int)a.size();
        st.assign(4 * n + 5, INF);
        build(1, 0, n - 1, a);
    }

    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            st[p] = a[l];
            return;
        }

        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m + 1, r, a);
        st[p] = min(st[2 * p], st[2 * p + 1]);
    }

    ll query(int p, int l, int r, int i, int j) {
        if (j < l || r < i) return INF;
        if (i <= l && r <= j) return st[p];

        int m = (l + r) / 2;
        return min(query(2 * p, l, m, i, j), query(2 * p + 1, m + 1, r, i, j));
    }

    void update(int p, int l, int r, int idx, ll val) {
        if (l == r) {
            st[p] = val;
            return;
        }

        int m = (l + r) / 2;
        if (idx <= m) update(2 * p, l, m, idx, val);
        else update(2 * p + 1, m + 1, r, idx, val);

        st[p] = min(st[2 * p], st[2 * p + 1]);
    }

    ll query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }

    void update(int idx, ll val) {
        update(1, 0, n - 1, idx, val);
    }
};
```

4.4 Fenwick Tree

```
struct Fenwick {
    int n;
    vector<ll> bit;

    Fenwick() {}
    Fenwick(int n): n(n), bit(n + 1, 0) {}

    // add value to index idx (1-indexed)
    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += val;
        }
    }
};
```

```

    }
}

// sum of [1..idx]
ll pref_sum(int idx) const {
    ll ans = 0;
    for (; idx > 0; idx -= idx & -idx) {
        ans += bit[idx];
    }
    return ans;
}

ll range_sum(int l, int r) const {
    if (l > r) return 0;
    return pref_sum(r) - pref_sum(l - 1);
}
};

```

4.5 Count inversions (Fenwick + compression)

```

struct Fenwick {
    int n;
    vector<ll> bit;

    Fenwick(int n): n(n), bit(n + 1, 0) {}

    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += val;
        }
    }

    ll pref_sum(int idx) const {
        ll ans = 0;
        for (; idx > 0; idx -= idx & -idx) {
            ans += bit[idx];
        }
        return ans;
    }
};

ll count_inversions_fenwick(vector<int> a) {
    if (a.empty()) return 0;

    vector<int> comp = a;
    sort(comp.begin(), comp.end());
    comp.erase(unique(comp.begin(), comp.end()), comp.end());

    int m = (int)comp.size();
    Fenwick fw(m);

    ll inv = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        int x = (int)(lower_bound(comp.begin(), comp.end(), a[i]) - comp.begin()) + 1;
        inv += fw.pref_sum(x - 1);
        fw.add(x, 1);
    }

    return inv;
}

```

4.6 K-th query on frequency SegTree

```

struct SegTreeFreq {
    int n;
    vector<int> st;

```

```

    SegTreeFreq(int n): n(n), st(4 * n + 5, 0) {}

    void add(int p, int l, int r, int idx, int delta) {
        if (l == r) {
            st[p] += delta;
            return;
        }

        int m = (l + r) / 2;
        if (idx <= m) add(2 * p, l, m, idx, delta);
        else add(2 * p + 1, m + 1, r, idx, delta);

        st[p] = st[2 * p] + st[2 * p + 1];
    }

    void add(int idx, int delta) {
        add(1, 0, n - 1, idx, delta);
    }

    // returns index of k-th element (1-indexed k), or -1 if k is invalid
    int kth(int p, int l, int r, int k) const {
        if (k <= 0 || k > st[p]) return -1;
        if (l == r) return l;

        int m = (l + r) / 2;
        if (st[2 * p] >= k) return kth(2 * p, l, m, k);
        return kth(2 * p + 1, m + 1, r, k - st[2 * p]);
    }

    int kth(int k) const {
        return kth(1, 0, n - 1, k);
    }
};

```

4.7 Binary search on SegTree

```

struct SegTreeMax {
    int n;
    vector<ll> st;

    SegTreeMax(const vector<ll>& a) {
        n = (int)a.size();
        st.assign(4 * n + 5, 0);
        build(1, 0, n - 1, a);
    }

    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            st[p] = a[l];
            return;
        }

        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m + 1, r, a);
        st[p] = max(st[2 * p], st[2 * p + 1]);
    }

    void update(int p, int l, int r, int idx, ll val) {
        if (l == r) {
            st[p] = val;
            return;
        }

        int m = (l + r) / 2;
        if (idx <= m) update(2 * p, l, m, idx, val);
        else update(2 * p + 1, m + 1, r, idx, val);
    }

```

```

    st[p] = max(st[2 * p], st[2 * p + 1]);
}

// first index with value >= k, or -1 if it does not exist
int first_ge(int p, int l, int r, ll k) const {
    if (st[p] < k) return -1;
    if (l == r) return l;

    int m = (l + r) / 2;
    if (st[2 * p] >= k) return first_ge(2 * p, l, m, k);
    return first_ge(2 * p + 1, m + 1, r, k);
}

int first_ge(ll k) const {
    return first_ge(1, 0, n - 1, k);
}

// consume k from first element >= k
int consume_first_ge(vector<ll>& a, ll k) {
    int idx = first_ge(k);
    if (idx == -1) return -1;

    a[idx] -= k;
    update(1, 0, n - 1, idx, a[idx]);
    return idx;
}
};

```

```

    st[p] = {val, val, val, val};
    return;
}

int m = (l + r) / 2;
if (idx <= m) update(2 * p, l, m, idx, val);
else update(2 * p + 1, m + 1, r, idx, val);

st[p] = merge_node(st[2 * p], st[2 * p + 1]);
}

void update(int idx, ll val) {
    update(1, 0, n - 1, idx, val);
}

ll max_subarray_sum() const {
    return st[1].best;
}
};

```

4.8 Subarray Sum Queries

```

struct Node {
    ll sum, pref, suff, best;
};

struct SegTreeMaxSubarray {
    int n;
    vector<Node> st;

    SegTreeMaxSubarray(const vector<ll>& a) {
        n = (int)a.size();
        st.assign(4 * n + 5, {0, 0, 0, 0});
        build(1, 0, n - 1, a);
    }

    Node merge_node(const Node& L, const Node& R) const {
        Node ans;
        ans.sum = L.sum + R.sum;
        ans.pref = max(L.pref, L.sum + R.pref);
        ans.suff = max(R.suff, R.sum + L.suff);
        ans.best = max({L.best, R.best, L.suff + R.pref});
        return ans;
    }

    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            st[p] = {a[l], a[l], a[l], a[l]};
            return;
        }

        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m + 1, r, a);
        st[p] = merge_node(st[2 * p], st[2 * p + 1]);
    }

    void update(int p, int l, int r, int idx, ll val) {
        if (l == r) {

```